

Hybrid Adaptive Crowd Simulation Using Noise-Based Coordination and Local Data Assimilation

CMPE 49G - Fundamentals of Particle-based Simulations, Boğaziçi University

Duran Kaan Altın - 2020400108

Enes Sait Besler - 2020400159

Abstract

Crowd simulation has a practical tension at its center. A model should be cheap enough to run many agents, but it should still respond when a local part of the crowd changes because of congestion, sensor observations, or risk. This paper studies two approaches that address different sides of this problem. The first uses Perlin noise as a coordination signal for large groups of non-player agents. Smooth noise fields can control movement parameters, action timing, and spatial events without requiring every agent to communicate with nearby agents. The second uses particle filtering for real-time crowd simulation, where an agent-based model is corrected with noisy observations. The noise-based method is scalable and reproducible, but it is not naturally tied to live observations. The particle filter is better suited for state estimation, but it becomes expensive as the crowd state grows.

Based on this comparison, we propose a hybrid adaptive crowd simulation framework. Most background agents are controlled by Perlin-like fields. Agents near sensors, exits, congestion, or danger zones are handled by a local correction layer based on particle filtering or a similar estimator. The local estimate then feeds back into the surrounding field, so the global controller is no longer completely static. This Phase-1 paper presents the motivation, background, proposed architecture, planned methodology, and possible extensions. It does not report experimental results.

Keywords: crowd simulation, Perlin noise, particle filter, data assimilation, agent-based simulation, evacuation, Voronoi diagram, adaptive fields

1. Introduction

Crowd simulation is used in games, virtual environments, evacuation planning, transportation hubs, and public-space monitoring. These applications do not ask for the same kind of realism. A game crowd may only need to look believable from a distance. In an evacuation or monitoring setting, the simulation also needs to stay close to what is happening in the environment.

This difference creates a trade-off between scale and correction. A procedural model can move many agents cheaply, but it may ignore local changes. A data-driven model can correct its state using observations, but it may become too expensive when it tries to estimate every agent in the scene. Our project starts from this trade-off and asks whether the two ideas can be combined instead of treated as alternatives.

The first reference we study is the work of Xu and Verbrugge on using Perlin noise as an AI

coordinator [1]. Their main idea is to use continuous noise fields not only for visual generation, but also for behavioral control. Because nearby points in a Perlin field have similar values, agents that are close to each other can behave in a locally coherent way without direct communication.

The second reference is the work of Malleson et al. on real-time crowd simulation with a particle filter [2,3]. Their approach treats crowd simulation as a data assimilation problem. Instead of trusting a single model prediction, the particle filter keeps several possible model states, compares them with observations, and resamples the states that better match the measurements.

The framework proposed in this paper combines these directions. Perlin-like fields provide a cheap background coordination layer. A correction layer, described mainly through particle filtering, is applied only in critical regions such as sensor areas, exits, bottlenecks, or danger zones. The goal is to keep the main simulation scalable while still allowing local adaptation where accuracy matters most.

2. Background

2.1 Crowd Simulation as an Agent-Based Problem

In an agent-based crowd simulation, each person or non-player character is represented as an individual entity. An agent usually has a position, velocity, target, speed, local perception, and behavioral state. The global crowd pattern emerges from the updates of many such agents.

A compact state description is

$$s_i(t) = (x_i(t), v_i(t), g_i(t), b_i(t)),$$

where $x_i(t)$ is position, $v_i(t)$ is velocity, $g_i(t)$ is the current goal, and $b_i(t)$ is the behavioral state of agent i . In a pedestrian setting, the goal may be an exit. In a game setting, it may be an activity region, patrol route, or event.

A simple position update has the form

$$x_i(t + \Delta t) = x_i(t) + v_i(t)\Delta t.$$

The important part is how $v_i(t)$ is chosen. It can depend on the goal direction, local interactions, obstacle avoidance, and random variation. Small local decisions can change the global pattern. For example, a few agents slowing down near a door can form a queue, and that queue can affect agents that are still far behind it. A useful model must therefore handle local variation without making every update too expensive.

2.2 Relation to Particle-Based Simulation

This topic fits the scope of particle-based simulation because the crowd is represented as a set of interacting entities. Crowd agents can be viewed as particles whose motion is affected by fields,

local interaction rules, and stochastic decisions. This connects the project to simulation ideas such as random walks, vector fields, diffusion, cellular automata, and Monte Carlo methods.

The probabilistic side is also central. Particle filtering relies on random variables, probability distributions, noisy measurements, and Bayesian updating. These concepts are directly related to the observation-based correction layer proposed later in the paper.

3. Perlin Noise as a Crowd Coordination Layer

3.1 Motivation

Large virtual environments often need many non-player agents. If every agent runs detailed decision logic or checks all nearby agents, the cost can grow quickly. At the other extreme, if agents use independent random numbers, the result may look noisy and disconnected. If they all follow the same rule, the crowd can look synchronized and artificial.

Perlin noise gives a useful middle ground. It produces smooth random fields: nearby positions receive related values, but the field still changes over space and time. This makes it possible to coordinate many agents through shared fields rather than through direct agent-to-agent communication.

For a 2D environment, a Perlin-like field can be written as

$$N(x, y, t),$$

where x and y are spatial coordinates and t is time. An agent samples the field at its current position. Since nearby agents sample nearby points, their sampled values are related but not identical.

3.2 Movement Parameterization

One direct use of noise is movement control. Two separate fields can define heading and speed:

$$\theta_i(t) = \Theta(x_i(t), t),$$

$$u_i(t) = U(x_i(t), t),$$

where Θ is a heading field and U is a speed field. The resulting velocity can be written as

$$v_i(t) = u_i(t) \begin{bmatrix} \cos(\theta_i(t)) \\ \sin(\theta_i(t)) \end{bmatrix}.$$

Using this value directly can still create sudden changes. A simple smoothing rule is

$$\theta_i(t + \Delta t) = \alpha \theta_i(t) + (1 - \alpha) \Theta(x_i(t), t),$$

where α controls inertia. A larger α makes the agent keep its previous direction more strongly. This is not meant to model exact human decision-making. It is better understood as a background coordination mechanism for coherent large-scale motion.

3.3 Activation Timing

Noise can also control when agents start or stop actions. For example, an activation probability can be defined as

$$P(a_i(t) = 1) = h(N_a(x_i(t), t)),$$

where N_a is an activation field and h maps the field value to a probability. If the field is high in one region, agents in that region are more likely to start the action. If it is low, they are less active.

This avoids two common problems: all agents acting at exactly the same time, or all agents acting independently with no visible structure.

3.4 Spatial Events and Environment Features

Noise fields can define spatial properties such as density, danger, rarity, event type, faction, or biome. In practice, separate fields should be used for unrelated properties. If density and danger share the same field, for example, high-density regions may always become dangerous even when that relationship was not intended.

Another useful property is reproducibility. Perlin fields are seedable, so the same seed can generate the same pattern again. This is helpful for debugging and for controlled experiments, because the same crowd setup can be rerun after changing only one part of the model.

3.5 Limitations

The weakness of the Perlin approach is that it does not automatically use observations. A pre-generated field can produce smooth behavior, but it does not know that a real crowd has become congested near an exit or that a new obstacle has appeared. It is scalable, but not naturally data-driven.

This limitation motivates adding a correction layer. The correction layer should be used only where it is needed; otherwise, the model loses the computational advantage that made the Perlin layer attractive in the first place.

4. Particle Filtering for Real-Time Crowd Simulation

4.1 Data Assimilation Problem

Malleson et al. treat crowd simulation as a real-time data assimilation problem [2,3]. A normal agent-based model predicts how the crowd moves. If the prediction is never corrected, however, it can drift away from the real system. A particle filter reduces this drift by updating the simulation with observations.

In a simple setting, the real state is unknown and observations are noisy:

$$y_t = H(x_t) + \eta_t,$$

where x_t is the true state, H maps the state to observable quantities, and η_t is measurement noise.

In their paper, Malleson et al. use a simplified pedestrian model called StationSim. Agents move through a station-like environment from entrance to exit. The particle filter keeps many possible versions of the model and updates them when observations arrive.

4.2 Particle Filter Steps

A particle filter represents uncertainty with a set of particles:

$$\{X_t^{(1)}, X_t^{(2)}, \dots, X_t^{(M)}\},$$

where each $X_t^{(j)}$ is one possible crowd state and M is the number of particles.

The first step is prediction. Each particle is moved forward using the crowd model:

$$X_t^{(j)} = f(X_{t-1}^{(j)}) + \epsilon_t,$$

where f is the model update function and ϵ_t is process noise.

The second step is observation. In an identical-twin experiment, observations can be generated from a known pseudo-truth by adding noise:

$$Y_t = X_t^{true} + \eta_t.$$

The third step is weighting. A particle closer to the observation receives a higher weight. A Gaussian likelihood can be written as

$$w_t^{(j)} \propto \exp\left(-\frac{\|Y_t - H(X_t^{(j)})\|^2}{2\sigma^2}\right),$$

where σ represents observation noise.

The final step is resampling. Low-weight particles are removed and high-weight particles are copied. This keeps the particle set concentrated around more likely states.

4.3 Strengths

Particle filtering is useful because it can handle nonlinear and non-Gaussian systems. Crowd motion is rarely perfectly linear. Small interactions can change later paths; for example, a faster pedestrian passing a slower one on the left or right may affect later interactions near a door.

The method also gives a direct representation of uncertainty. Instead of producing only one predicted state, it keeps several possible states and updates their likelihoods as observations arrive.

4.4 Limitations

The main problem is scalability. If the state includes the positions of N agents in 2D, the state dimension is already $2N$. If velocities, goals, or behavioral parameters are included, the dimension becomes even larger.

With a fixed number of particles, the update may be manageable for small systems. For large crowds, the number of particles needed to represent the full state can grow very quickly. This leads to particle degeneracy, where most particles have almost zero weight, and particle deprivation, where resampling leaves too little diversity in the particle set.

For this reason, applying a particle filter to the whole crowd is not a practical starting point for our project. The hybrid design uses particle filtering only in local critical regions.

5. Proposed Hybrid Adaptive Crowd Simulation Framework

5.1 Main Idea

The proposed model uses different levels of detail for different parts of the crowd. Most agents are controlled by a Perlin-like field, which keeps the simulation cheap and scalable. Agents near critical regions are handled by a correction layer. This layer can be a particle filter, but the architecture does not require particle filtering specifically.

The separation is

background agents $\rightarrow$ Perlin field control,

critical agents $\rightarrow$ local correction layer.

The correction layer does not replace the Perlin field. Instead, it updates or biases the field locally. This is the part that makes the field adaptive rather than purely procedural.

5.2 Agent Categories

At each time step, an agent is classified as either background or critical. Let $R_c(t)$ be the set of critical regions. These regions may include sensors, exits, congested areas, danger zones, or high-uncertainty areas. Agent i is critical if

$$x_i(t) \in R_c(t).$$

A binary variable can represent the classification:

$$C_i(t) = \begin{cases} 1, & x_i(t) \in R_c(t), \\ 0, & \text{otherwise.} \end{cases}$$

If $C_i(t) = 0$, the agent follows the background field. If $C_i(t) = 1$, the agent is promoted to the correction layer. This promotion can be temporary; after the agent leaves the critical region, it can return to background control.

5.3 Background Motion Model

For background agents, movement is defined by a weighted combination of goal direction, noise direction, and local avoidance:

$$d_i(t) = \lambda_1 d_i^{goal}(t) + \lambda_2 d_i^{noise}(t) + \lambda_3 d_i^{avoid}(t),$$

where $d_i^{goal}(t)$ points toward the current target or exit, $d_i^{noise}(t)$ comes from the Perlin-like field, $d_i^{avoid}(t)$ represents local density or collision avoidance, and $\lambda_1, \lambda_2, \lambda_3$ are weights.

The normalized direction is used to update the position:

$$x_i(t + \Delta t) = x_i(t) + s_i(t) \frac{d_i(t)}{\|d_i(t)\|} \Delta t,$$

where $s_i(t)$ is the agent speed.

This model keeps the crowd directed while still allowing variation. The goal component moves agents toward exits or targets. The noise component prevents the motion from becoming completely deterministic. The avoidance component reacts to nearby density.

5.4 Local Correction Layer

For critical agents, the correction layer estimates a local state:

$$X_t^c = \{x_i(t), v_i(t), g_i(t)\}_{i \in C},$$

where C is the set of critical agents.

If a particle filter is used, it runs on this local state rather than on the entire crowd. This reduces the effective dimensionality. The correction layer can use noisy observations from sensors such as cameras, overhead tracking, Wi-Fi/Bluetooth localization, or gate counters.

In a synthetic experiment, observations can be generated from pseudo-truth:

$$Y_t^c = X_t^{c,true} + \eta_t.$$

The resulting local estimate is then used to update the nearby crowd field.

5.5 Feedback from Correction Layer to Perlin Field

The feedback step is the key part of the hybrid model. The correction layer produces local information about density, velocity, congestion, or uncertainty. This information modifies the field in nearby regions.

An adapted heading field can be written as

$$\Theta'(x, t) = \Theta(x, t) + \gamma A(x, t),$$

where $\Theta(x, t)$ is the original heading field, $A(x, t)$ is the adaptation term, and γ controls feedback strength.

The adaptation term can include avoidance from danger, redirection around congestion, or stronger bias toward exits:

$$A(x, t) = \mu_1 A^{danger}(x, t) + \mu_2 A^{exit}(x, t) + \mu_3 A^{density}(x, t).$$

This allows the global field to react to local observations without applying a full particle filter to every agent.

5.6 Dynamic Critical Zones

A critical zone does not need to be a fixed physical object. It can represent risk, congestion, or uncertainty. If density increases near an exit, for example, the affected region can expand.

A simple radius update is

$$r_c(t + \Delta t) = r_c(t) + \beta U(t),$$

where $U(t)$ is a risk or uncertainty signal and β is a sensitivity parameter. In an implementation, $U(t)$ may depend on maximum density, queue length, speed drop, or estimator variance.

This is useful in evacuation scenarios because congestion does not stay at a single point. It spreads and affects nearby agents.

6. Application-Oriented Design Considerations

The main contribution of this Phase-1 work is the hybrid adaptive architecture. The topics in this section are possible design choices for Phase-2 experiments. They make the proposal more concrete, but they are not replacements for the main model.

6.1 Evacuation Scenario with a Second Door

One possible test case is an evacuation environment where one door already exists and a second door is added. This scenario does not change the central idea of the project. It gives us a measurable

way to evaluate whether local adaptation helps near bottlenecks.

Let Door 1 be fixed:

$$d_1 = \text{fixed.}$$

The second door is a candidate location on a valid boundary:

$$d_2 \in \partial\Omega_{\text{valid}}.$$

For each candidate d_2 , the simulation can measure evacuation metrics. A possible objective function is

$$J(d_2) = w_1 T_{\text{last}}(d_2) + w_2 \bar{T}_{\text{evac}}(d_2) + w_3 \rho_{\text{max}}(d_2) + w_4 I_{\text{door}}(d_2),$$

where T_{last} is the time when the last agent exits, $\bar{T}_{\text{evac}}$ is the average evacuation time, ρ_{max} is the maximum local density, and I_{door} is the imbalance between exit usage.

This second-door case is useful because exits are natural critical regions. The correction layer can be activated near doors and bottlenecks, while the rest of the crowd still uses Perlin-based motion.

6.2 Voronoi Diagrams for Exit Assignment

Voronoi diagrams can support the second-door experiment by giving an initial spatial assignment of agents to exits. With two doors, the room can be partitioned based on nearest exit.

For doors d_1 and d_2 , the Voronoi region of door d_j is

$$V_j = \{x : \|x - d_j\| \leq \|x - d_k\|, \forall k \neq j\}.$$

This gives a first estimate of which agents are naturally closer to each door. It can also show whether the second door actually divides the crowd or whether most agents still prefer the original door.

Normal Voronoi partitioning only considers distance. In evacuation, congestion and door capacity also matter. A weighted version can be used:

$$V_j = \{x : \alpha\|x - d_j\| + \beta Q_j \leq \alpha\|x - d_k\| + \beta Q_k\},$$

where Q_j is the queue or congestion near door j . With this form, an agent may choose a farther door if the closer door is too crowded.

In this project, Voronoi diagrams are a supporting tool for exit assignment and interpretation. They are not the main crowd model.

6.3 Simplex-Based Optimization Possibilities

Simplex-based methods may also be useful in Phase-2, but the word “simplex” has two meanings here.

The first meaning is the classical simplex algorithm for linear programming. Door placement itself is not usually a linear programming problem because evacuation time comes from simulation and nonlinear agent interactions. However, a linear programming subproblem can be formed for regional flow allocation.

Suppose the environment is divided into regions. Let x_{ij} be the number of agents assigned from region i to door j . A simple linear objective is

$$\min \sum_i \sum_j c_{ij} x_{ij}$$

subject to

$$\sum_j x_{ij} = n_i,$$

$$\sum_i x_{ij} \leq Cap_j,$$

$$x_{ij} \geq 0,$$

where c_{ij} is assignment cost, n_i is the number of agents in region i , and Cap_j is the capacity of door j . This subproblem can be solved with the linear programming simplex algorithm.

The second meaning is the Nelder-Mead simplex method. This is a derivative-free optimization method. It may be useful if the second-door location is treated as a continuous design variable and the objective value is obtained from simulation. Since the simulation objective may be noisy and non-differentiable, derivative-free optimization may be more practical than a gradient-based method.

For this Phase-1 proposal, simplex methods remain optional evaluation or optimization tools. They are not part of the core hybrid simulation architecture.

6.4 Alternatives to Particle Filtering

Particle filtering is useful because it handles nonlinear and non-Gaussian uncertainty. It is still expensive in high-dimensional systems, so the correction layer should be modular.

One alternative is the Ensemble Kalman Filter. It uses an ensemble of model states, but updates them using mean and covariance information. It may scale better than a particle filter in some high-dimensional cases. Its weakness is that it works best when uncertainty is not too far from Gaussian.

Another alternative is density-based filtering. Instead of estimating each individual agent, the model estimates density over grid cells. This may be more realistic if sensors provide heat maps, crowd counts, or regional densities rather than exact trajectories.

A third option is a cellular automata-based evacuation model. The environment is discretized into cells, and agents move according to local transition rules. This is efficient and works naturally with grid-based densities, but it can introduce artifacts because motion is restricted by the grid.

These alternatives do not remove particle filtering from the project. They show that the correction layer can be implemented in more than one way. For Phase-1, the particle filter remains the main reference method. For Phase-2, a simpler density-based correction may also be tested if time is limited.

7. Planned Methodology for Phase-2

This paper does not include experimental results. Phase-2 will implement the proposed hybrid model and compare it with simpler baselines.

7.1 Simulation Environment

The initial environment will be a 2D rectangular or station-like domain. Agents will start from random or structured initial positions and move toward exits or target regions. The simulation will include local density effects and a noise-based movement component.

The environment may include one or two exits, sensor zones, a central congestion or danger region, optional obstacles, and different crowd sizes.

7.2 Compared Models

The planned comparison includes three models:

1. **Pure Perlin model:** all agents follow Perlin-like fields with goal-directed motion.
2. **Particle-filter-assisted model:** a local particle filter is applied in critical regions.
3. **Hybrid adaptive model:** background agents use Perlin fields, critical agents are corrected, and the local estimate modifies the field.

If time allows, a density-based correction model can be added as an alternative to the particle filter.

7.3 Metrics

The evaluation will use both computational and behavioral metrics.

Computational metrics include runtime per frame, frames per second, memory usage, number of agents, number of tracked agents, and number of particles if particle filtering is used.

Behavioral and evacuation metrics include average evacuation time, last-agent evacuation time, maximum local density, density near exits, door usage balance, and adaptation time after a congestion or danger event.

For state estimation, position RMSE may be used:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\hat{x}_i - x_i^{true}\|^2},$$

where $\hat{x}_i$ is the estimated position and x_i^{true} is the pseudo-truth position.

7.4 Expected Behavior

The pure Perlin model is expected to be the fastest, but it may not react well to local changes. A full particle filter should be more accurate for small systems, but its cost should increase quickly as the number of agents grows. The hybrid model is expected to sit between these two cases: lower cost than full particle filtering and better local adaptation than pure Perlin control.

These are hypotheses, not results. They will be tested in Phase-2.

8. Discussion

The proposed framework is based on a simple modeling choice: not all agents need the same level of detail. In many crowd simulations, most agents only need to behave plausibly. More accurate estimation is needed mainly near exits, sensors, bottlenecks, and danger zones.

The Perlin layer provides scalable background motion. The correction layer provides local adaptation. The feedback mechanism connects them, so the two methods are not just running side by side.

The second-door evacuation scenario is useful because it turns the framework into a concrete test case. Still, it is only one possible application. Voronoi diagrams and simplex-based optimization are also supporting tools rather than replacements for the main model.

A limitation of the current proposal is that the exact correction layer has not been implemented yet. Particle filtering is the main reference method, but alternatives may be more practical depending on the final simulation scale. Another limitation is parameter tuning. The weights for Perlin motion, exit direction, avoidance, and adaptation must be selected carefully. If the feedback is too weak, the field will not adapt. If it is too strong, the motion may become unstable or unnatural.

9. Conclusion

This paper presented a Phase-1 proposal for hybrid adaptive crowd simulation. We studied two approaches: Perlin-noise-based coordination and particle-filter-based data assimilation. The Perlin approach is efficient and useful for large background crowds, but it lacks real-time grounding. The particle filter can correct model drift using observations, but it is expensive for large crowds.

The proposed framework combines these ideas. Most agents follow a Perlin-like background field. Agents near critical regions are handled by a local correction layer. The correction layer then updates the surrounding field, allowing the crowd to adapt to congestion, risk, or sensor observations.

We also discussed possible Phase-2 extensions. An evacuation scenario with one fixed door and a candidate second door can be used as a future test case. Voronoi diagrams can help with initial exit assignment. Simplex-based methods may support optimization or flow allocation. Alternatives to particle filtering, such as Ensemble Kalman Filtering or density-based correction, may also be considered.

Phase-2 will focus on implementation, comparison with baselines, and evaluation using runtime, density, evacuation time, and state-estimation metrics.

References

- [1] K. Xu and C. Verbrugge, “(Perlin) Noise as AI Coordinator,” arXiv:2602.18947, 2026.
- [2] N. Malleson, K. Minors, L.-M. Kieu, J. A. Ward, A. A. West, and A. Heppenstall, “Simulating Crowds in Real Time with Agent-Based Modelling and a Particle Filter,” arXiv:1909.09397, 2019.
- [3] N. Malleson, K. Minors, L.-M. Kieu, J. A. Ward, A. A. West, and A. Heppenstall, “Simulating Crowds in Real Time with Agent-Based Modelling and a Particle Filter,” *Journal of Artificial Societies and Social Simulation*, vol. 23, no. 3, 2020.
- [4] D. Shiffman, *The Nature of Code*. The Nature of Code Foundation.
- [5] H. B. Yilmaz, “CMPE 49G: Fundamentals of Particle-based Simulations,” course materials, Boğaziçi University, 2026.